U24AI006

ASSIGNMENT -07

The 8 queen's problem is stated as: Place 8 queens on a chessboard such that no two queens attack each other. Considering this problem, complete the following tasks considering a valid heuristic function with justification.

Q1 Generate 50 random initial boards. Run steepest -ascent hill climbing on each and record initial heuristic value, final heuristic value, number of steps taken and status solved or fail. Prove the presence of a local minimum.

Q2 Repeat the above experiment on first choice, random restart and simulated annealing variants of hill climbing and compare your results.

CODE(1):

```
import random

def random_board():
    return [random.randint(0, 7) for _ in range(8)]

def heuristic(board):
    h = 0
    for i in range(8):
        for j in range(i + 1, 8):
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(i - j):
                h += 1
    return h

def steepest_ascent(board):
    steps = 0
    current = board[:]
    current_h = heuristic(current)

    while True:
        best = current[:]
        best_h = current_h

        for col in range(8):
            original_row = current[col]
            for row in range(8):
                if row != original_row:
                    current[col] = row
                    h = heuristic(current)
                    if h < best_h:
                        best_h = h
                        best = current[:]
```

```
            current[col] = original_row

        if best_h >= current_h:
            return current_h, steps, ("Solved" if current_h == 0 else "Fail")

        current = best[:]
        current_h = best_h
        steps += 1

print(f"{'Run':<5} | {'Initial H':<10} | {'Final H':<8} | {'Steps':<6} |
{'Status'}")
print("-" * 50)

fail_count = 0
success_count = 0

for i in range(50):
    board = random_board()
    initial_h = heuristic(board)
    final_h, steps, status = steepest_ascent(board)

    if status == "Fail":
        fail_count += 1
    else:
        success_count += 1

    print(f"{i+1:<5} | {initial_h:<10} | {final_h:<8} | {steps:<6} | {status}")

print("\n--- Statistics ---")
print(f"Total Successes: {success_count}")
print(f"Total Fails (Local Minima): {fail_count}")
print(f"Success Rate: {(success_count/50)*100}%")
```

OUTPUT:

U24AI006

```
 QUESTION 1.py"
Run    | Initial H   | Final H   | Steps   | Status
-----------------------------------------------------
1      | 6           | 1         | 3       | Fail
2      | 6           | 3         | 1       | Fail
3      | 10          | 1         | 3       | Fail
4      | 7           | 1         | 4       | Fail
5      | 8           | 1         | 4       | Fail
6      | 6           | 0         | 3       | Solved
7      | 8           | 0         | 4       | Solved
8      | 9           | 2         | 3       | Fail
9      | 4           | 2         | 2       | Fail
10     | 3           | 0         | 3       | Solved
11     | 10          | 1         | 4       | Fail
12     | 4           | 1         | 2       | Fail
13     | 5           | 2         | 2       | Fail
14     | 10          | 2         | 2       | Fail
15     | 9           | 0         | 5       | Solved
16     | 7           | 0         | 5       | Solved
17     | 7           | 1         | 3       | Fail
18     | 9           | 2         | 5       | Fail
19     | 10          | 1         | 4       | Fail
20     | 6           | 1         | 3       | Fail
21     | 11          | 1         | 4       | Fail
22     | 12          | 1         | 5       | Fail
23     | 6           | 1         | 3       | Fail
24     | 6           | 1         | 3       | Fail
25     | 12          | 1         | 5       | Fail
26     | 7           | 1         | 4       | Fail
27     | 7           | 1         | 3       | Fail
28     | 9           | 1         | 4       | Fail
29     | 12          | 0         | 4       | Solved
30     | 6           | 2         | 2       | Fail
31     | 9           | 1         | 4       | Fail
32     | 9           | 1         | 3       | Fail
33     | 10          | 2         | 4       | Fail
34     | 4           | 0         | 2       | Solved
35     | 9           | 2         | 3       | Fail
36     | 6           | 1         | 3       | Fail
37     | 10          | 2         | 3       | Fail
38     | 7           | 1         | 3       | Fail
39     | 8           | 1         | 3       | Fail
40     | 7           | 1         | 4       | Fail
41     | 10          | 1         | 4       | Fail
42     | 6           | 1         | 3       | Fail
43     | 5           | 2         | 2       | Fail
44     | 6           | 0         | 4       | Solved
45     | 6           | 3         | 1       | Fail
46     | 8           | 1         | 3       | Fail
47     | 12          | 1         | 5       | Fail
48     | 11          | 1         | 5       | Fail
49     | 7           | 2         | 3       | Fail
50     | 8           | 1         | 3       | Fail

--- Statistics ---
Total Successes: 8
Total Fails (Local Minima): 42
```

CODE(2):

```
import random
import math

def heuristic(board):
    conflicts = 0
    n = len(board)
    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(i - j):
                conflicts += 1
    return conflicts

def generate_random_board(n=8):
    return [random.randint(0, n - 1) for _ in range(n)]

def get_neighbors(board):
    neighbors = []
    n = len(board)
    for col in range(n):
        for row in range(n):
            if board[col] != row:
                new_board = list(board)
                new_board[col] = row
                neighbors.append(new_board)
    return neighbors

def first_choice_hill_climbing(board):
    current = list(board)
    current_h = heuristic(current)
    steps = 0
    n = len(current)

    while True:
        found = False
        attempts = 0
        max_attempts = n * (n - 1)

        while attempts < max_attempts:
            col = random.randint(0, n - 1)
            row = random.randint(0, n - 1)
            if row == current[col]:
                attempts += 1
                continue
            neighbor = list(current)
```

```
            neighbor[col] = row
            h = heuristic(neighbor)
            attempts += 1
            if h < current_h:
                steps += 1
                current = neighbor
                current_h = h
                found = True
                break

        if not found:
            break
        if current_h == 0:
            break

    return current, current_h, steps

def random_restart_hill_climbing(max_restarts=100):
    total_steps = 0
    restart_count = 0

    while restart_count <= max_restarts:
        board = generate_random_board()
        current = list(board)
        current_h = heuristic(current)
        steps = 0

        while True:
            neighbors = get_neighbors(current)
            best_neighbor = None
            best_h = current_h

            for neighbor in neighbors:
                h = heuristic(neighbor)
                if h < best_h:
                    best_h = h
                    best_neighbor = neighbor

            if best_neighbor is None:
                break

            steps += 1
            current = best_neighbor
            current_h = best_h
```

```
            if current_h == 0:
                break

        total_steps += steps

        if current_h == 0:
            return current, current_h, total_steps, restart_count

        restart_count += 1

    return current, current_h, total_steps, restart_count

def simulated_annealing(board, initial_temp=4.0, cooling_rate=0.995,
min_temp=0.0001):
    current = list(board)
    current_h = heuristic(current)
    temperature = initial_temp
    steps = 0
    n = len(current)

    while temperature > min_temp:
        col = random.randint(0, n - 1)
        row = random.randint(0, n - 1)
        while row == current[col]:
            row = random.randint(0, n - 1)

        neighbor = list(current)
        neighbor[col] = row
        neighbor_h = heuristic(neighbor)
        delta = neighbor_h - current_h

        if delta < 0:
            accept = True
        else:
            probability = math.exp(-delta / temperature)
            accept = random.random() < probability

        steps += 1

        if accept:
            current = neighbor
            current_h = neighbor_h

        temperature *= cooling_rate
```

```
        if current_h == 0:
            break

    return current, current_h, steps


fc_results = []
fc_solved = 0

rr_results = []
rr_solved = 0

sa_results = []
sa_solved = 0

for trial in range(1, 51):
    board = generate_random_board()
    initial_h = heuristic(board)

    final_board, final_h, num_steps = first_choice_hill_climbing(board)
    status = "Solved" if final_h == 0 else "Failed"
    if final_h == 0:
        fc_solved += 1
    fc_results.append((trial, initial_h, final_h, num_steps, status))

    final_board, final_h, total_steps, restarts = random_restart_hill_climbing()
    status = "Solved" if final_h == 0 else "Failed"
    if final_h == 0:
        rr_solved += 1
    rr_results.append((trial, final_h, total_steps, restarts, status))

    board = generate_random_board()
    initial_h = heuristic(board)
    final_board, final_h, num_steps = simulated_annealing(board)
    status = "Solved" if final_h == 0 else "Failed"
    if final_h == 0:
        sa_solved += 1
    sa_results.append((trial, initial_h, final_h, num_steps, status))


print("First Choice Hill Climbing")
print(f"{'Trial':<8} {'Initial H':<12} {'Final H':<10} {'Steps':<8} {'Status'}")
print("-" * 50)
for trial, init_h, fin_h, steps, status in fc_results:
    print(f"{trial:<8} {init_h:<12} {fin_h:<10} {steps:<8} {status}")
```

```
fc_avg_steps = sum(r[3] for r in fc_results) / 50
print(f"\nSolved: {fc_solved}/50  |  Success Rate: {(fc_solved/50)*100:.1f}%  |
Avg Steps: {fc_avg_steps:.1f}")


print("\nRandom Restart Hill Climbing")
print(f"{'Trial':<8} {'Final H':<10} {'Total Steps':<14} {'Restarts':<10}
{'Status'}")
print("-" * 50)
for trial, fin_h, steps, restarts, status in rr_results:
   print(f"{trial:<8} {fin_h:<10} {steps:<14} {restarts:<10} {status}")
rr_avg_steps = sum(r[2] for r in rr_results) / 50
rr_avg_restarts = sum(r[3] for r in rr_results) / 50
print(f"\nSolved: {rr_solved}/50  |  Success Rate: {(rr_solved/50)*100:.1f}%  |
Avg Steps: {rr_avg_steps:.1f}  |  Avg Restarts: {rr_avg_restarts:.1f}")


print("\nSimulated Annealing")
print(f"{'Trial':<8} {'Initial H':<12} {'Final H':<10} {'Steps':<8} {'Status'}")
print("-" * 50)
for trial, init_h, fin_h, steps, status in sa_results:
   print(f"{trial:<8} {init_h:<12} {fin_h:<10} {steps:<8} {status}")
sa_avg_steps = sum(r[3] for r in sa_results) / 50
print(f"\nSolved: {sa_solved}/50  |  Success Rate: {(sa_solved/50)*100:.1f}%  |
Avg Steps: {sa_avg_steps:.1f}")


print("\nComparison Summary")
print(f"{'Algorithm':<25} {'Solved':<10} {'Success %':<12} {'Avg Steps'}")
print("-" * 55)
print(f"{'First Choice HC':<25} {fc_solved:<10} {(fc_solved/50)*100:<12.1f}
{fc_avg_steps:.1f}")
print(f"{'Random Restart HC':<25} {rr_solved:<10} {(rr_solved/50)*100:<12.1f}
{rr_avg_steps:.1f}")
print(f"{'Simulated Annealing':<25} {sa_solved:<10} {(sa_solved/50)*100:<12.1f}
{sa_avg_steps:.1f}")
```

CODE(2):

First Choice Hill Climbing

Trial Initial H Final H Steps Status

---

1 12 2 6 Failed

2	12	2	5	Failed
3	13	2	7	Failed
4	9	1	7	Failed
5	12	3	6	Failed
6	6	1	4	Failed
7	7	1	4	Failed
8	9	2	5	Failed
9	10	0	7	Solved
10	7	2	3	Failed
11	9	1	7	Failed
12	5	1	3	Failed
13	5	2	2	Failed
14	8	2	5	Failed
15	8	3	4	Failed
16	3	1	2	Failed
17	7	2	4	Failed
18	8	2	6	Failed
19	9	2	6	Failed
20	5	1	3	Failed
21	9	2	6	Failed
22	4	2	2	Failed
23	9	2	6	Failed
24	11	1	6	Failed
25	5	3	2	Failed
26	7	2	3	Failed
27	7	2	4	Failed
28	8	2	5	Failed
29	5	4	1	Failed
30	10	3	5	Failed

31	8	2	5	Failed
32	7	2	5	Failed
33	10	2	5	Failed
34	6	2	3	Failed
35	7	4	3	Failed
36	7	0	4	Solved
37	4	3	1	Failed
38	6	1	4	Failed
39	8	4	2	Failed
40	6	2	3	Failed
41	12	1	8	Failed
42	8	2	5	Failed
43	7	3	4	Failed
44	8	2	6	Failed
45	8	1	6	Failed
46	7	3	2	Failed
47	5	3	2	Failed
48	9	1	6	Failed
49	4	2	2	Failed
50	9	1	7	Failed

Solved: 2/50 | Success Rate: 4.0% | Avg Steps: 4.4

Random Restart Hill Climbing

Trial	Final H	Total Steps	Restarts	Status
1	0	22	6	Solved
2	0	35	11	Solved
3	0	39	12	Solved

4	0	26	7	Solved
5	0	14	3	Solved
6	0	62	20	Solved
7	0	47	15	Solved
8	0	13	2	Solved
9	0	4	0	Solved
10	0	20	7	Solved
11	0	11	2	Solved
12	0	9	2	Solved
13	0	22	7	Solved
14	0	81	23	Solved
15	0	18	4	Solved
16	0	21	6	Solved
17	0	4	0	Solved
18	0	6	1	Solved
19	0	27	6	Solved
20	0	3	0	Solved
21	0	5	0	Solved
22	0	13	3	Solved
23	0	32	10	Solved
24	0	23	5	Solved
25	0	27	7	Solved
26	0	16	4	Solved
27	0	14	4	Solved
28	0	5	2	Solved
29	0	5	0	Solved
30	0	40	12	Solved
31	0	13	3	Solved
32	0	2	0	Solved

33	0	21	7	Solved
34	0	14	3	Solved
35	0	8	1	Solved
36	0	17	4	Solved
37	0	56	16	Solved
38	0	12	2	Solved
39	0	10	2	Solved
40	0	40	11	Solved
41	0	26	7	Solved
42	0	10	1	Solved
43	0	18	5	Solved
44	0	15	3	Solved
45	0	23	6	Solved
46	0	9	1	Solved
47	0	12	3	Solved
48	0	7	1	Solved
49	0	37	11	Solved
50	0	7	2	Solved

Solved: 50/50 | Success Rate: 100.0% | Avg Steps: 20.4 | Avg Restarts: 5.4

Simulated Annealing

Trial	Initial H	Final H	Steps	Status
1	5	0	596	Solved
2	8	0	1280	Solved
3	5	1	2115	Failed
4	11	0	368	Solved

5	8	0	728	Solved
6	6	0	1557	Solved
7	11	0	1125	Solved
8	10	0	1178	Solved
9	7	0	621	Solved
10	6	0	345	Solved
11	8	0	566	Solved
12	10	0	637	Solved
13	7	0	978	Solved
14	8	0	600	Solved
15	7	0	683	Solved
16	9	0	697	Solved
17	5	0	886	Solved
18	8	0	1621	Solved
19	9	0	492	Solved
20	11	0	1044	Solved
21	8	0	1047	Solved
22	8	0	816	Solved
23	11	0	806	Solved
24	13	0	610	Solved
25	8	0	1090	Solved
26	7	0	665	Solved
27	12	0	477	Solved
28	10	1	2115	Failed
29	5	0	626	Solved
30	7	0	1613	Solved
31	11	0	1466	Solved
32	6	0	1155	Solved
33	5	0	618	Solved

34	9	0	612	Solved
35	6	0	579	Solved
36	7	0	846	Solved
37	7	0	602	Solved
38	6	0	521	Solved
39	8	0	403	Solved
40	12	0	675	Solved
41	3	0	749	Solved
42	8	0	737	Solved
43	8	0	536	Solved
44	8	0	675	Solved
45	3	0	1433	Solved
46	8	0	900	Solved
47	5	0	455	Solved
48	8	0	1542	Solved
49	5	0	384	Solved
50	5	1	2115	Failed

Solved: 47/50 | Success Rate: 94.0% | Avg Steps: 899.7

Comparison Summary

Algorithm	Solved	Success %	Avg Steps
First Choice HC	2	4.0	4.4
Random Restart HC	50	100.0	20.4
Simulated Annealing	47	94.0	899.7